Extending your generated clients is useful, but during testing it's common to also want to substitute your generated client by using interfaces. Interfaces let you substitute fake or mock versions of a service so that your tests aren't calling the API for real, as you learned in chapter 8. NSwag can help with this process by automatically generating an `IFruitClient` interface that the `FruitClient` implements.

Finally, providing a base address where the API is hosted makes sense on the face of it. But as we discussed in chapter 9, primitive constructor arguments such as `string` and `int` don't play well with dependency injection. Given that `HttpClient` contains a `BaseAddress` property, you can configure NSwag to *not* require that the base address be passed as a constructor argument and instead set it on the `HttpClient` type directly. This approach helps in dependency injection (DI) scenarios, as you'll see when we discuss `IHttpClientFactory` in chapter 33.

These three seemingly unrelated options are all configured in NSwag in the same way: by adding an `Options` element to the `<OpenApiReference>` element in your project file. The options are provided as command-line switches and must be provided on one line, without line breaks. The switches for the three settings described are

- `/UseBaseUrl:false`—When `false`, NSwag removes the `baseUrl` parameter from the generated client's constructor and instead relies on `HttpClient` to have the correct base address. It defaults to `true`.
- `/GenerateClientInterfaces:true`—When `true`, NSwag generates an interface for the client, containing all the endpoints. The generated client implements this interface. It defaults to `false`.